

Rapport : Affectation des étudiants en Master

Implémentation de Gale-Shapley et résolution par programmation linéaire.

Jesus Alejandro Gomez Urzua
Nadhir Ben Said

10 mai 2026

Table des matières

1	Introduction	1
2	Algorithme de Gale-Shapley	2
2.1	Optimisation avec des structures de données	2
2.2	Adaptation pour l'exécution côté parcours	2
2.3	Jeux de tests et analyse de performances	2
3	Programmation Linéaire	4
3.1	Maximisation de l'utilité minimale des étudiants	4
3.2	Maximisation de l'utilité totale	4
3.3	Optimisation sous contrainte de rang	5
4	Analyse comparative	5

1 Introduction

Ce projet porte sur l'étude et la résolution du problème de l'affectation des étudiants au sein des différents parcours de Master. L'objectif est de coupler n étudiants à m formations en tenant compte de leurs préférences respectives, tout en respectant les capacités d'accueil limitées propres à chaque Master. L'enjeu majeur réside dans l'implémentation d'algorithmes capables de satisfaire simultanément plusieurs critères :

1. Stabilité : garantir qu'aucun couple (étudiant, Master) n'ait intérêt à rompre l'affectation actuelle pour se choisir mutuellement.
2. Efficacité : maximiser le bien-être total des participants selon une approche utilitariste.
3. Équité : assurer un niveau de satisfaction minimal pour tous les agents suivant une approche égalitariste.

Pour ce faire, nous proposons dans un premier temps une implémentation de l'algorithme de Gale-Shapley, qui garantit l'obtention d'un mariage stable sans toutefois assurer l'efficacité ou l'équité de la solution. Dans un second temps, nous utiliserons la programmation linéaire afin d'optimiser les critères d'efficacité et d'équité, indépendamment des contraintes de stabilité.

2 Algorithme de Gale-Shapley

L'algorithme de Gale-Shapley repose sur une stratégie de type « propositions et rejets » qui garantit l'obtention d'un mariage stable. La version implémentée dans ce projet adapte le modèle vu en cours pour intégrer les capacités d'accueil des Masters. Dans cette configuration, un parcours accepte une proposition tant qu'il dispose de places vacantes. Une fois sa capacité atteinte, il ne valide une nouvelle candidature que si l'étudiant proposant est mieux classé que l'étudiant le moins préféré parmi ceux déjà acceptés.

2.1 Optimisation avec des structures de données

Afin de garantir une complexité efficace, nous avons couplé chaque opération élémentaire de l'algorithme à une structure de données adaptée :

1. **Identification des étudiants libres** : L'utilisation d'une `deque` permet d'extraire un étudiant et de le réinsérer en cas de rejet en $\mathcal{O}(1)$.
2. **Sélection du prochain parcours** : Un tableau d'entiers mémorise pour chaque étudiant l'indice du prochain Master à solliciter dans sa liste de préférences, garantissant une sélection en $\mathcal{O}(1)$.
3. **Évaluation du rang** : Le pré-calcul d'une matrice des rangs en $\mathcal{O}(n \times m)$ permet à chaque Master de connaître en temps constant la position d'un candidat dans son classement.
4. **Gestion des admissions** : Chaque Master gère ses étudiants admis via un tas binaire. Cette structure permet d'identifier l'étudiant le moins préféré en $\mathcal{O}(1)$ et de mettre à jour l'affectation en $\mathcal{O}(\log(k))$, où k est la capacité maximale d'un parcours.

Ainsi, notre implémentation atteint une complexité globale de $\mathcal{O}(n \times m \log(k))$

2.2 Adaptation pour l'exécution côté parcours

Il est important de noter que cet algorithme n'est pas symétrique : il favorise systématiquement le groupe qui émet les propositions. En l'occurrence, cette version est *étudiant-optimale*, offrant à chaque étudiant la meilleure affectation possible parmi tous les mariages stables, tout en étant *parcours-pessimale*.

Dans la version où les Masters proposent aux étudiants, l'implémentation se rapproche davantage du modèle abordé en cours et devient plus simple. En effet, la gestion des priorités par un tas binaire devient superflue : chaque étudiant ne pouvant être affecté qu'à un seul Master, il lui suffit de comparer toute nouvelle offre à son affectation actuelle pour l'accepter ou la rejeter. Pour gérer les capacités d'accueil, chaque Master est inséré dans la `deque` autant de fois qu'il possède de places disponibles. Ainsi, la somme des capacités étant égale au nombre d'étudiants, dans cette version la complexité de $\mathcal{O}(n^2)$.

2.3 Jeux de tests et analyse de performances

La fiabilité de l'implémentation a été vérifiée par un jeu de tests via la bibliothèque `unittest`. Divers scénarios d'affectation ont été élaborés manuellement et stockés dans un fichier `JSON` incluant les résultats attendus. Ces tests couvrent l'algorithme de Gale-Shapley (variantes étudiant et parcours) ainsi que la fonction de détection des paires instables.

Pour confronter les performances observées à la complexité théorique, nous avons mesuré le temps d'exécution et le nombre d'itérations pour des instances allant de 200 à 2000 étudiants.

L'algorithme a d'abord été évalué sur des matrices de préférences aléatoires et uniformes. En fixant initialement le nombre de parcours à $m = 10$, nous observons que le temps d'exécution et le nombre d'itérations adoptent un comportement quasi-linéaire (Figure 1). Ce résultat est cohérent avec la complexité théorique en $\mathcal{O}(n \times m \log k)$, qui devient $\mathcal{O}(n \log n)$ lorsque m est constante.

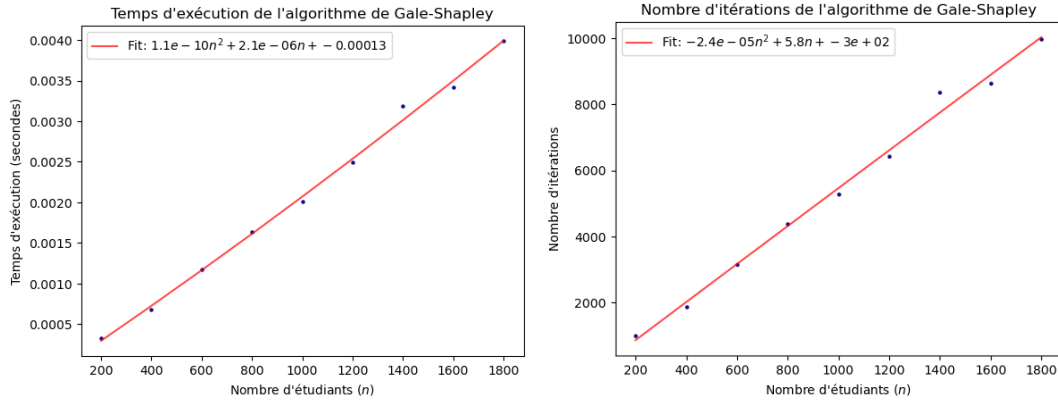


FIGURE 1 – Exécution de l'algorithme Gale-Shapley avec 10 parcours.

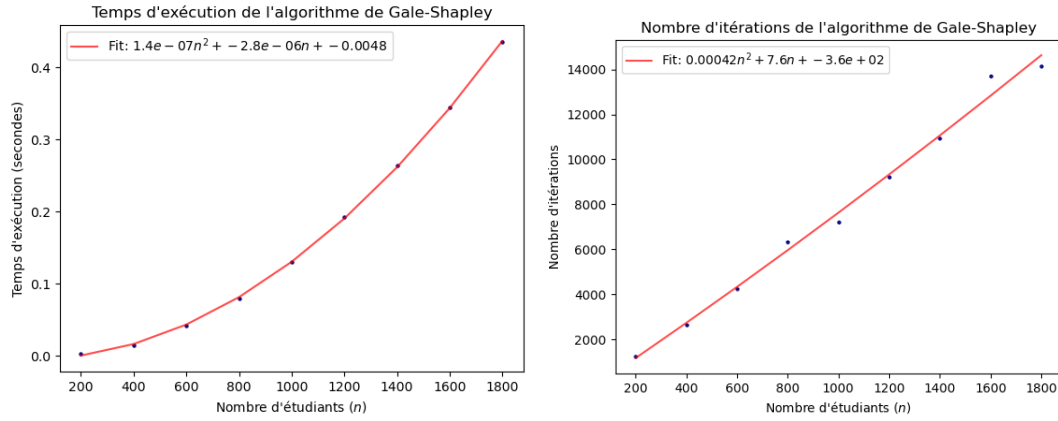


FIGURE 2 – Exécution de l'algorithme Gale-Shapley pour des données aléatoires.

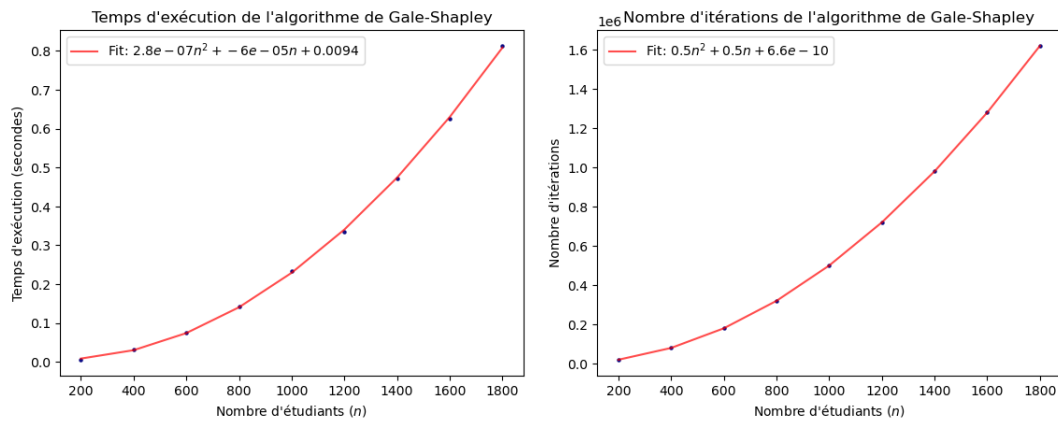


FIGURE 3 – Exécution de l'algorithme Gale-Shapley avec des données forçant le pire cas.

Afin d'étudier le comportement asymptotique de l'algorithme, nous avons ensuite fait varier simultanément le nombre d'étudiants et de parcours en fixant $m = n$. Les résultats présentés en Figure 2 confirment un temps d'exécution quadratique. Toutefois, le nombre d'itérations croît plus lentement que prévu. Cette observation corrobore les travaux de Wilson (1972), qui démontrent que, sous l'hypothèse de préférences indépendantes et uniformes, le nombre moyen de propositions est en $\mathcal{O}(n \log n)$. La divergence observée entre le temps d'exécution et le nombre d'itérations s'explique par la phase de pré-calcul des rangs, dont la complexité reste structurellement en $\mathcal{O}(n \times m)$ indépendamment de la distribution des préférences.

Finalement, nous avons soumis l'implémentation à un scénario de pire cas (Figure 3). Dans cette configuration, le temps de calcul et le nombre d'itérations suivent tous deux un comportement quadratique, confirmant ainsi la borne supérieure théorique de l'algorithme.

3 Programmation Linéaire

Cette seconde partie du projet explore l'utilisation de la programmation linéaire pour résoudre le problème d'affectation sous différents critères, tels que l'efficacité et l'équité.

Pour quantifier la satisfaction des agents, nous définissons l'utilité par les scores de Borda. Afin de simplifier nos calculs, dans nos programmes linéaires, nous utiliserons directement le rang dans la matrice de préférences. Ainsi, maximiser l'utilité revient à minimiser le rang. On note $b_{j,i}$ le rang du i -ème étudiant dans le j -ème parcours, et $c_{i,j}$ le rang du j -ème parcours dans la liste de préférences du i -ème étudiant et C_j la capacité du j -ème parcours.

3.1 Maximisation de l'utilité minimale des étudiants

Pour garantir une certaine équité, nous cherchons à minimiser le mécontentement de l'étudiant le plus mal loti. Nous introduisons une variable auxiliaire z représentant le rang maximal atteint par un étudiant.

Variables :

- $x_{i,j} \in \{0,1\}$: vaut 1 si l'étudiant i est affecté au Master j , 0 sinon.
- $z \in \mathbb{N}$: le rang maximal observé.

Fonction objectif : $\min z$

Contraintes :

$$\begin{aligned} \sum_{j=1}^m x_{i,j} &= 1, & \forall i \in \{1, \dots, n\} \\ \sum_{j=1}^m c_{i,j} x_{i,j} &\leq z, & \forall i \in \{1, \dots, n\} \\ \sum_{i=1}^n x_{i,j} &\leq C_j, & \forall j \in \{1, \dots, m\} \end{aligned}$$

3.2 Maximisation de l'utilité totale

Le modèle précédent omettait les préférences des établissements. Nous cherchons désormais à maximiser l'efficacité globale en optimisant la somme des utilités des étudiants et des parcours (approche utilitariste).

Variables :

- $x_{i,j} \in \{0,1\}$: vaut 1 si l'étudiant i est affecté au Master j , 0 sinon.

Étudiant	GS côté étudiants	GS côté parcours	Max min util	Max util totale	Max util totale $k = 5$
0	5	5	9	8	8
1	6	6	6	5	5
2	9	9	2	4	4
3	9	9	9	9	9
4	1	1	1	1	1
5	0	0	4	0	9
6	8	8	7	7	7
7	7	7	0	0	0
8	3	3	5	9	6
9	2	2	8	2	2
10	4	4	3	4	3
11	4	4	4	3	4
12	0	0	0	6	0

TABLE 1 – Récapitulatif des affectations par méthode pour chaque étudiant.

Fonction objectif : $\min \sum_{i=1}^n \sum_{j=1}^m x_{i,j}(b_{i,j} + c_{i,j})$

Contraintes :

$$\begin{aligned} \sum_{j=1}^m x_{i,j} &= 1, & \forall i \in \{1, \dots, n\} \\ \sum_{i=1}^n x_{i,j} &\leq C_j, & \forall j \in \{1, \dots, m\} \end{aligned}$$

3.3 Optimisation sous contrainte de rang

Cette variante maximise l'utilité totale tout en garantissant à chaque étudiant une affectation parmi ses k premiers vœux.

Variables :

— $x_{i,j} \in \{0, 1\}$: vaut 1 si l'étudiant i est affecté au Master j , 0 sinon.

Fonction objectif : $\min \sum_{i=1}^n \sum_{j=1}^m x_{i,j}(b_{i,j} + c_{i,j})$

Contraintes :

$$\begin{aligned} \sum_{j=1}^m x_{i,j} &= 1, & \forall i \in \{1, \dots, n\} \\ \sum_{i=1}^n x_{i,j} &\leq C_j, & \forall j \in \{1, \dots, m\} \\ \sum_{j=1}^m c_{i,j} x_{i,j} &\leq k, & \forall i \in \{1, \dots, n\} \end{aligned}$$

Dans nos tests, cette affectation était possible uniquement pour les $k \geq 5$

4 Analyse comparative

Afin d'évaluer les différentes méthodes étudiées, nous les avons appliquées au jeu de données fourni. Les affectations obtenues sont consignées dans le Tableau 1.

Métrique	GS côté étudiants	GS côté parcours	Max min util	Max util totale	Max util totale $k = 5$
Somme des utilités	203	203	178	213	201
Utilité moyenne (étudiants)	7.84	7.84	7.61	7.15	7.84
Utilité moyenne (parcours)	7.76	7.76	6.07	9.23	7.61
Utilité minimale (étudiants)	4	4	5	0	5
Utilité minimale (parcours)	0	0	0	0	0
Paires instables	0	0	9	7	5

TABLE 2 – Analyse comparative des indicateurs de performance.

Nous avons analysé ces résultats selon des indicateurs de stabilité (nombre de paires instables), d'équité (utilité minimale des étudiants) et d'efficacité (somme totale des utilités). Le Tableau 2 synthétise ces mesures, permettant ainsi d'apprécier les compromis entre stabilité, satisfaction globale et équité propres à chaque modèle.

Nous pouvons constater que les deux variantes de l'algorithme de Gale-Shapley aboutissent à la même affectation, démontrant que le mariage stable est unique pour cette instance. Le programme linéaire maximisant l'utilité totale atteint la meilleure efficacité globale au détriment de la stabilité et de l'équité, puisqu'au moins un étudiant obtient son pire choix. À l'inverse, l'approche égalitariste protège les étudiants les plus défavorisés mais s'accompagne d'une chute de l'efficacité et de la plus forte instabilité. Dans ce contexte, l'optimisation sous contrainte de rang se révèle être un excellent compromis. Elle garantit un niveau de satisfaction minimal et limite les paires instables tout en offrant une efficacité globale presque équivalente à celle du mariage stable.

Références

Wilson, L. B. (1972), 'An analysis of the stable marriage assignment algorithm', *BIT Numerical Mathematics* **12**(4), 569–575.